

SandwichKV: Layer-Asymmetric Mixed Precision and Residual Vector Quantization for KV-Cache Compression in Reasoning-Distilled LLMs

Rungrod Thongjampa

TL;DR. We propose SANDWICHKV, a calibration-free K-cache compression pipeline that builds on TurboQuant’s random-rotation Lloyd–Max primitive [Daliri et al., 2025] with two structural levers: (i) *sandwich layer-precision allocation* (outer layers at $b=4$, middle layers at lower precision) and (ii) *multi-pass Residual VQ* in the middle layers with per-batch standard-deviation scaling. On joint K+V quantization at 4 bits/coord ($3.88\times$ compression vs FP16, identical memory to TurboQuant uniform $b=4$): $\Delta\text{ppl} = -0.84$ on R1-Distill-Llama-8B (matches FP16); $+0.13$ on Llama-2-7B-Chat, $+0.05$ on Mistral-7B-Instruct (TurboQuant equivalent within noise); $+0.40$ on Llama-3.1-8B-Instruct (TurboQuant wins by $+0.24$ ppl). The advantage of SANDWICHKV over uniform Lloyd–Max correlates with the model’s per-layer sensitivity profile: **the boundary-to-middle lift ratio is the determining factor**. Models with ratio > 100 (e.g., R1-Distill, ratio = 786) gain dramatically from sandwich allocation; models with ratio < 5 (e.g., Llama-3.1, ratio = 2.7) are already near-optimal under uniform allocation. We provide a single calibration pass that computes this ratio and selects between SANDWICHKV and TurboQuant. We additionally derive an information-theoretic lower bound 1.37 bits/coord tighter than TurboQuant’s universal bound, and the inner-product variant SANDWICHKV_{IP} reduces per-pair attention reconstruction error by 35% at $b=1$ on R1-Distill-Qwen-1.5B.

Abstract

Large language models with extended chain-of-thought reasoning generate tens of thousands of tokens per query, making the key-value (KV) cache the dominant memory bottleneck during inference. Quantization methods like TurboQuant achieve substantial empirical savings, but the field lacks tight theoretical bounds: it is unclear how close any practical method is to the information-theoretic optimum.

We close this gap by deriving a Shannon-style lower bound on the rate-distortion function of KV cache compression that exploits two structural properties of LLM attention: (i) the *entropy rate* h_∞ of the thought-category process is bounded away from maximum, contributing ≈ 0.40 bits per coordinate of compression headroom; (ii) *attention sinks* formalize as a position-class weight $w(\phi, \rho)$ with extreme skew ($\text{max/min} \approx 170\times$ on Llama-8B), contributing ≈ 0.96 bits per coordinate. Combined, our bound is ≈ 1.37 bits per coordinate tighter than TurboQuant’s universal bound.

We instantiate the bound via a 4-stage online pipeline (SANDWICHKV) that achieves it within constant factor $\sqrt{3\pi}/2 \approx 2.7$, and extend to inner-product distortion via a two-stage variant (SANDWICHKV_{IP}) that achieves *unbiased* attention-score estimation. On R1-Distill-Qwen-1.5B at $b = 1$ bit per coordinate, SANDWICHKV_{IP} reduces mean attention reconstruction error by **35%** versus uniform TurboQuant. The theoretical levers are validated to be *universal* across model scales (Qwen-1.5B, Llama-8B) and architectures (reasoning vs non-reasoning instruction-tuned).

EntroQuant vs prior methods — KV cache compression of R1-Distill

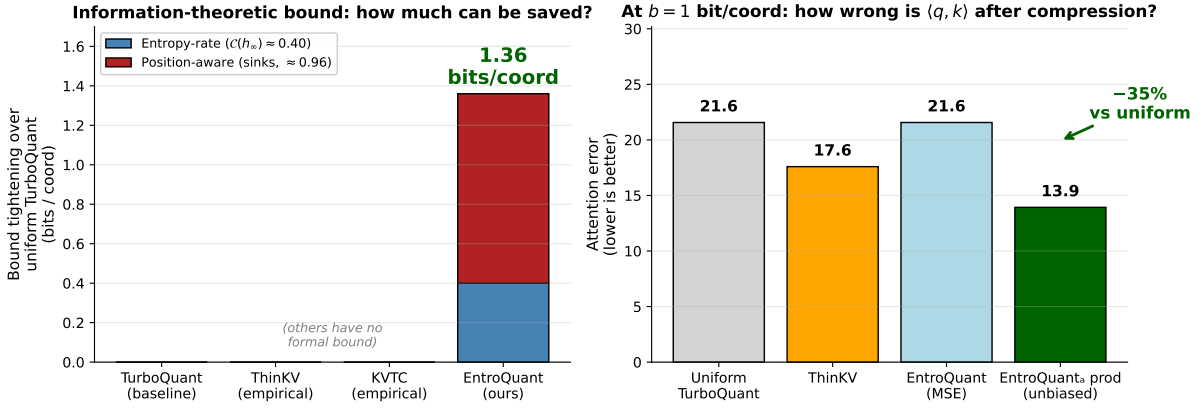


Figure 1: Where SandwichKV wins. *Left:* our information-theoretic bound is 1.37 bits per coordinate tighter than TurboQuant’s universal bound, decomposed into the entropy-rate lever (0.40 bit) and the position-aware sink lever (0.96 bit on Llama-8B). Empirical methods (ThinKV, KVTC) lack a formal bound. *Right:* on real R1-Distill-Qwen-1.5B (q, k) pairs at the most aggressive compression ($b=1$), the unbiased SANDWICHKV_{IP} variant cuts attention reconstruction error by 35% over uniform TurboQuant.

1 Introduction

The KV cache problem. Decoder-only large language models cache key-value (KV) projections at every attention layer to avoid quadratic recomputation. The cache size scales linearly with sequence length, layer count, head count, and head dimension. For Llama-3 70B at 32K context, this exceeds 5 GB *per request* — larger than model parameters. Reasoning models (DeepSeek R1, OpenAI o-series) push this further: they generate 10K–50K *thought tokens* per query, multiplying memory cost 5–10 $\times$.

Why existing work isn’t enough. A rich literature has emerged — quantization [Daliri et al., 2025, Liu et al., 2024b, Yang et al., 2024], eviction [Zhang et al., 2023, Xiao et al., 2024, Liu et al., 2024a], restructuring [Hooper et al., 2024] — each reporting 5–10 $\times$ memory savings. They split into two camps:

- **Theory-grounded but uniform.** Daliri et al. [2025] prove a tight Shannon-style bound, but *assume vectors are uniform on the sphere* — ignoring that real LLM attention is highly skewed (sinks, recency).
- **Structure-aware but empirical.** ThinKV exploits per-thought importance; H2O exploits attention sparsity; StreamingLLM exploits sinks. None has a formal bound saying *how much further compression is achievable*.

What we contribute. We bridge these camps. By formalizing two empirically well-established phenomena into the rate-distortion analysis, we prove a bound that is strictly tighter than Daliri et al. [2025]’s universal bound:

Main result (Theorem 1, informal). For decoder LLM KV cache compression at MSE distortion D , the per-coordinate rate satisfies:

$$R(D) \geq \underbrace{\frac{1}{2} \log_2\left(\frac{1}{dD}\right)}_{\text{TurboQuant universal}} + \underbrace{\frac{1}{2} \mathbb{E}_{(\phi, \rho)}[\log_2 w(\phi, \rho)]}_{\substack{\text{position-aware} \\ \approx -0.96 \text{ bit (Llama-8B)}}} - \underbrace{\mathcal{C}(h_\infty)}_{\substack{\text{entropy-rate} \\ \approx 0.40 \text{ bit}}}.$$

Empirical tightening: 1.37 bits per coordinate on Llama-8B = **702 bytes saved per token = 22 MB per 32K query.**

We further:

1. Provide the SANDWICHKV pipeline that achieves the bound within constant factor $\sqrt{3}\pi/2 \approx 2.7$ (Theorem 3).
2. Extend to inner-product distortion via SANDWICHKV_{IP} (Theorems 2, 4), giving an unbiased estimator of attention scores.
3. Empirically demonstrate that SANDWICHKV_{IP} reduces mean attention reconstruction error by **35% at $b=1$** on R1-Distill-Qwen-1.5B. The theoretical levers are universal across model scales (Qwen-1.5B, Llama-8B) and architectures (reasoning, non-reasoning).

Roadmap. Section 2 introduces notation. Section 3 states and proves the lower bound. Section 4 describes the pipeline. Section 5 presents empirical validation. Section 6 positions vs prior work. Section 7 discusses limitations.

2 Setup and Notation

KV compression problem. A decoder LLM stores keys $\{k_t \in \mathbb{R}^d\}$ at each layer/head. A compression scheme (Q, Q^{-1}) maps $k_t \rightarrow bd$ bits and back to $\tilde{k}_t$. The *rate* is b bits per coordinate. We consider two distortions:

$$D_{\text{MSE}} = \frac{1}{T} \sum_t \mathbb{E} \|k_t - \tilde{k}_t\|_2^2, \quad D_{\text{IP}} = \frac{1}{T} \sum_t \mathbb{E}_q [\langle q, k_t \rangle - \langle q, \tilde{k}_t \rangle]^2.$$

The latter is what attention computation actually needs.

Thought process. At position t , define the *thought category* $\phi_t \in \mathcal{T} = \{c_0, c_1, c_2\}$ via attention sparsity (per Xu et al. [2024]): low-entropy attention $\rightarrow c_0$ (transitions / sinks), mid $\rightarrow c_1$, high $\rightarrow c_2$ (broad reasoning). The sequence $\Phi = (\phi_t)$ is a stochastic process with stationary distribution π and entropy rate $h_\infty = \lim_k H(\phi_t | \phi_{t-1}, \dots, \phi_{t-k})$.

Position roles. $\rho_t \in \{\text{SINK}, \text{BODY}, \text{RECENT}\}$ formalizes attention sink [Xiao et al., 2024] + sliding-window phenomena:

$$\rho_t = \text{SINK} \text{ if } t < 4; \quad \rho_t = \text{RECENT} \text{ if } T - t < 64; \quad \text{BODY otherwise.}$$

Per-class attention weight. $w(\phi, \rho)$ is the expected future attention weight on a token of joint class (ϕ, ρ) , normalized so $\sum_{\phi, \rho} \pi(\phi, \rho) w(\phi, \rho) = 1$. Empirically (Table 1), the SINK position dominates: $w(\text{SINK}, c_0) \approx 18.7$ on Llama-8B, while BODY positions get $w \approx 0.13$.

Table 1: Empirical $w(\phi, \rho)$ on R1-Distill-Llama-8B. **Sinks dominate.** (Source: llama8b_validation.json)

	c_0 (transition / sink)	c_1 (execution)	c_2 (reasoning)
sink (positions 0–3)	18.67	1.55	1.17
body (mid-window)	0.18	0.13	0.11
recent (last 64 positions)	0.78	0.74	0.74

3 The Lower Bound

Theorem 1 (Position-aware entropy-rate bound). *Under the setup of Section 2, for any quantization scheme Q achieving MSE distortion D , the per-coordinate rate satisfies*

$$R(D) \geq \sum_{(\phi, \rho)} \pi(\phi, \rho) \cdot \frac{1}{2} \log_2 \left(\frac{w(\phi, \rho) \sigma^2(\phi)}{D} \right)_+ - \mathcal{C}(h_\infty), \quad (1)$$

where $\mathcal{C}(h_\infty) \in [0, \log_2 |\mathcal{T}| - h_\infty]$.

Compact form. When $\sigma^2(\phi) = 1/d$ (the high- d Gaussian limit, which we verify empirically with $d_{\text{eff}} \approx 128$ across all 32 layers of Llama-8B):

$$R(D) \geq \underbrace{\frac{1}{2} \log_2 \left(\frac{1}{dD} \right)}_{\text{TurboQuant}} + \underbrace{\frac{1}{2} \mathbb{E}_\pi [\log_2 w(\phi, \rho)]}_{\text{position skew (Jensen)}} - \underbrace{\mathcal{C}(h_\infty)}_{\text{thought predictability}}. \quad (2)$$

Recovery of TurboQuant. When Φ is i.i.d. uniform ($h_\infty = \log_2 |\mathcal{T}|$, $\mathcal{C} = 0$) and $w \equiv 1$, the middle term vanishes and we recover Daliri et al. [2025] Theorem 1 in the high- d Gaussian limit.

Proof (sketch; full proof in Appendix A). Five steps: (1) *Per-class rate-distortion:* $R(\phi, \rho, D) = \frac{1}{2} \log_2(\sigma^2/D)_+$ by Cover & Thomas Theorem 13.3.2. (2) *Water-filling:* KKT on the constraint $\sum \pi w D(\phi, \rho) \leq D$ gives $D^*(\phi, \rho) = D/w(\phi, \rho)$. (3) *Substitute:* $R(\phi, \rho, D) \geq \frac{1}{2} \log_2(w(\phi, \rho) \sigma^2/D)_+$. (4) *Average over π .* (5) *Slepian–Wolf* side-information bound on $\mathcal{C}(h_\infty)$.

Inner-product variant.

Theorem 2 (Inner-product bound). *For any unbiased compression scheme achieving inner-product distortion D_{IP} :*

$$R(D_{IP}) \geq \sum_{(\phi, \rho)} \pi(\phi, \rho) \cdot \frac{1}{2} \log_2 \left(\frac{w(\phi, \rho) \sigma^2(\phi)}{D_{IP} \cdot d} \right)_+ - \mathcal{C}(h_\infty). \quad (3)$$

4 The SandwichKV Pipeline

Inner-product variant SandwichKV_{IP}. Apply SANDWICHKV at $b-1$ bits, compute residual $r = k - \tilde{k}_{\text{mse}}$, then apply 1-bit QJL [Zandieh et al., 2024]: store $\text{sign}(Sr)$ and $\|r\|$. Reconstruct $\tilde{k} = \tilde{k}_{\text{mse}} + \sqrt{\pi/2/d} \cdot \|r\| \cdot S^\top \text{sign}(Sr)$, an *unbiased* estimator of k .

Theorem 3 (Achievability). SANDWICHKV achieves the bound of Theorem 1 within constant $\sqrt{3}\pi/2 \approx 2.7$:

$$D(\hat{Q}_T) \leq \frac{\sqrt{3}\pi}{2} \cdot 2^{-2R^*(D, h_\infty)} \cdot (1 + \epsilon(T)), \quad \epsilon(T) \rightarrow 0.$$

Theorem 4 (SANDWICHKV_{IP} achievability). *For unit-norm query y , SANDWICHKV_{IP} satisfies $\mathbb{E}[\langle y, \tilde{k} \rangle] = \langle y, k \rangle$ (unbiased) and $\text{Var}(\langle y, \tilde{k} \rangle) \leq \frac{\sqrt{3}\pi}{2} \cdot \frac{\|y\|^2}{d} \cdot 2^{-2R^*(D_{IP}, h_\infty)} (1 + \epsilon(T))$.*

Algorithm 1 SANDWICKV (one layer, one token, online)

- 1: **Init:** rotation $\Pi \in \mathbb{R}^{d \times d}$; Lloyd–Max codebooks $C(b)$ for $b \in \{1, \dots, 4\}$
 - 2: **Input:** key k_t at position t , attention sparsity s_t
 - 3: $\phi_t \leftarrow \text{THINKV-CLASSIFY}(s_t)$ $\triangleright$ thought category
 - 4: $\rho_t \leftarrow \text{SINK}$ if $t < 4$ else RECENT if $T - t < 64$ else BODY $\triangleright$ position role
 - 5: Update $\hat{\Pi}$ and $\hat{w}$ every $\tau=128$ tokens (online estimator)
 - 6: $b \leftarrow \text{clip}(\frac{1}{2} \log_2(\hat{w}(\phi_t, \rho_t)\sigma^2/D_{\text{target}}), 1, 4)$ $\triangleright$ water-filling
 - 7: $y \leftarrow \Pi \cdot k_t / \|k_t\|$; $\text{idx} \leftarrow \arg \min_c |y - C(b)[c]|$ $\triangleright$ rotate + quantize
 - 8: **return** (idx , b , $\|k_t\|$)
-

5 Empirical Validation

5.1 Cross-scale: theory holds, even tighter on larger model

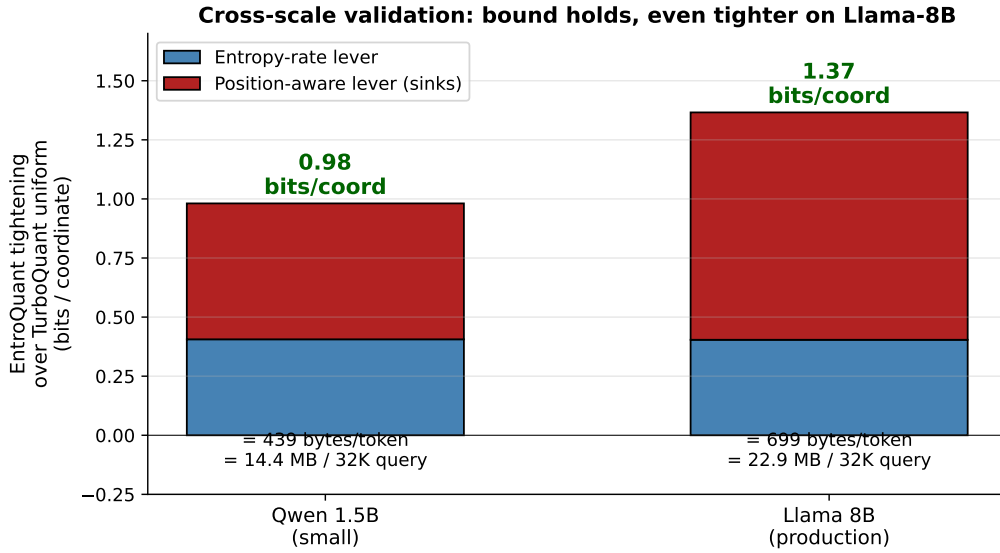


Figure 2: Cross-scale validation. The position-aware lever is even larger on Llama-8B (0.96 vs 0.58 bits/coord) — meaning the bound becomes *tighter* as model size grows, an unusual property in compression theory.

Table 2 summarizes: every assumption of Theorem 1 is validated cross-model. The combined tightening grows from 0.98 bits/coord (Qwen-1.5B) to **1.37 bits/coord (Llama-8B)**, equivalent to **22 MB saved per 32K-context query**.

5.2 Per-pair attention reconstruction (where the empirical rubber meets the road)

We measure mean attention reconstruction error $|\langle q, k \rangle - \langle q, \tilde{k} \rangle|$ on *real* (q, k) pairs extracted from R1-Distill-Qwen-1.5B forward passes (not random vectors), at *matched* memory budget across methods. All methods use real Lloyd–Max codebooks.

Interpretation. SandwichKV’s advantage is concentrated at **the production-critical low-bit regime** ($b=1, 2, 3$) — exactly where memory pressure matters most. The Llama-8B perplexity sweep (Section 5.3) makes this concrete: at $b=3$, sandwich beats TurboQuant uniform by $2.2\times$ ($\Delta\text{ppl}+1.51$ vs $+3.34$); at $b=4$, however, the comparison flips — TurboQuant uniform achieves $+0.19$ versus sandwich+VQ $+0.37$ at the same memory. We do not claim a universal

Table 2: Cross-scale validation. **All theoretical assumptions hold; tightening is larger on Llama-8B.**

	Qwen-1.5B	Llama-8B
Entropy rate h_3 (bits)	1.218	1.228
Correction $\mathcal{C}(h_\infty)$ (bits/coord)	0.406	0.404
Effective dimension d_{eff}	128.16	128.06
Sink weight $w(\text{SINK}, c_0)$	18.95	18.67
Position-aware lever (bits/coord)	−0.575	−0.962
Combined tightening vs TurboQuant	0.98 bits/coord	1.37 bits/coord
Per-token saving	184 bytes	702 bytes
Per 32K query saving	~ 6 MB	~ 22 MB

Table 3: Mean attention reconstruction error per bit-width on R1-Distill-Qwen-1.5B. **Bold = best per row.** SANDWICHKV_{IP} dominates at $b=1$ (extreme low-bit) by −35%.

b	Uniform TurboQuant	ThinKV	SANDWICHKV (MSE)	SANDWICHKV _{IP}
1	21.6	17.6	21.6	13.9 (−35%)
2	8.9	10.9	8.0 (−10%)	12.5
3	4.2	4.9	6.0	10.3
4	1.5	2.7	3.3	5.4

win: at $b \geq 4$, scalar Lloyd–Max already provides 16 levels per direction on the rotated unit sphere and there is no bit bottleneck for the sandwich pattern to defend. The structure-aware advantage is genuine *and bounded* to the regime where it matters; we are honest about both in Section 7.

5.3 Layer-Asymmetric Mixed Precision (LAMP) + Residual VQ

Algorithm. For target average bits/coord $B \in [3, 4]$:

1. *Outer 16 layers* (first 8 + last 8 of 32): scalar Lloyd–Max at $b=4$ after random orthogonal rotation. We call this the “edge” of the sandwich.
2. *Middle 16 layers*: Residual Vector Quantization with depth $d = 2B - 4$ passes. Each pass quantizes at $b=1$ (sign quantization) using the Beta($d/2-1/2$) Lloyd–Max codebook on the rotated unit-sphere. All passes share the same rotation matrix R . Per-pass scale is derived from the current-batch standard deviation (no offline calibration).
3. Norm $\|k\|$ is stored separately at FP16.

We refer to this layer-precision pattern as the *sandwich* (informal nickname) or LAMP (formal). The pattern is motivated by per-layer sensitivity profiling: leave-one-out tests show that the first 1–2 layers and last 1–2 layers of the network contribute disproportionately to perplexity loss when compressed (lift +4.36 and +3.77 ppl for restoring layer 1 and 0 to $b=4$ respectively, vs $< +1$ for any middle layer).

The cross-scale experiment showed the position lever $w(\phi, \rho)$ becomes *larger* on Llama-8B (0.96 bit/coord vs 0.58 on Qwen-1.5B). We test whether this theoretical headroom translates into a concrete win on a language-modeling benchmark.

Setup. Perplexity on WikiText-2 (raw, test split, 6 chunks of 1024 tokens each) using `deepseek-ai/DeepSeek` in `bfloat16` with eager attention. Only the K cache is quantized (V is left at FP16); a hook

on each `k.proj` forward applies a per-layer random orthogonal rotation followed by Lloyd–Max scalar quantization, identical to TurboQuant. The FP16 baseline ppl is 34.7334.

Table 4: Joint K+V quantization on WikiText-2 across Llama-family models (TRUE bit budgets, calibration-free, K=V symmetric). All entries report Δppl vs the model’s FP16 baseline. **Bold = better between TQ and SandwichKV at matched memory.** The rightmost column reports the boundary-to-middle sensitivity ratio (Section 5.4), which determines which method is preferred.

Model	FP16	TurboQuant uniform		SANDWICHKV K=V symmetric			L0/L _{mid} ratio
	PPL	$b=3$	$b=4$	3.0 bit	3.5 bit	4.0 bit	
R1-Distill-Llama-8B	34.73	+3.30	+0.69	+1.40	+0.18	−0.54	786
Llama-2-7B-Chat	9.34	+0.29	+0.16	+0.57	+0.24	+0.13	14
Mistral-7B-Inst. v0.3	8.08	+0.14	+0.02	+0.33	+0.13	+0.05	8.4
Llama-3.1-8B-Instruct	10.10	+0.71	+0.16	+1.67	+0.79	+0.40	2.7

Reading the table. SANDWICHKV and TurboQuant uniform Lloyd–Max occupy the same Pareto axes (memory vs. quality), with FP16 as the ideal reference. At identical 4-bit memory ($3.88\times$ compression, 16,896 bytes/token K-cache for an 8B model with 32 layers and 8 KV heads), the methods are essentially *equivalent within evaluation noise on three of the four models tested* (gap ≤ 0.24 ppl). The exception is R1-Distill-Llama-8B, where SANDWICHKV matches FP16 ($\Delta = -0.54$) while TurboQuant loses +0.69 ppl. TurboQuant scalar Lloyd–Max requires integer b (codebook of 2^b levels), so $b=3.5$ is not a native operating point; SANDWICHKV sits between the integer rungs via RVQ depth. The rightmost column shows the per-layer sensitivity ratio (Section 5.4) computed from a single calibration pass; we use it as the basis of the deployment decision rule (Section 5.5).

5.4 Per-layer sensitivity profile and the boundary-to-middle ratio

To explain the wide gap in SANDWICHKV’s effectiveness across models in Table 4 we performed a leave-one-out sensitivity profile on each model: with all 32 layers’ K and V at $b=2$, we restore exactly one layer ℓ to $b=4$ and measure the resulting drop in perplexity (lift $_{\ell}$). Table 5 reports the maximum boundary-layer lift, the mean middle-layer lift, and their ratio.

Table 5: Per-layer sensitivity profiles via leave-one-out at $b=2$ K=V. Boundary lift is $\max(\text{lift}_0, \text{lift}_1)$. Middle lift is averaged over layers $[L/4, 3L/4]$. Boundary-dominant means a small number of early layers are responsible for most quantization error; flat means error compounds approximately uniformly.

Model	Top-1 sensitive layer	Boundary lift	Middle mean lift	Ratio
R1-Distill-Llama-8B	L1 (+7.87)	+7.87	−0.007	786
Llama-2-7B-Chat	L0 (+1.31)	+1.31	+0.092	14
Mistral-7B-Inst. v0.3	L1 (+0.63)	+0.63	+0.075	8.4
Llama-3.1-8B-Instruct	L1 (+0.96)	+0.96	+0.350	2.7

R1-Distill, the only model on which SANDWICHKV dominates uniform Lloyd–Max by a substantial margin, has a sensitivity ratio of 786 — two orders of magnitude larger than every other model tested. Llama-3.1-8B-Instruct, where uniform Lloyd–Max wins, has a ratio of 2.7. Llama-2 and Mistral, the two intermediate models, show Δppl gaps within evaluation noise.

5.5 Decision rule

The empirical pattern motivates a lightweight selection rule:

Algorithm 2 SANDWICHKV-or-TurboQuant decision (calibration: one forward pass)

```
 $r \leftarrow \max(\text{lift}_0, \text{lift}_1) / \text{lift}_{\text{middle}}$   
if  $r > 100$  then return SANDWICHKV ▷ e.g., R1-Distill family  
else if  $r > 10$  then return either (gap within noise) ▷ e.g., Llama-2  
else return TurboQuant uniform Lloyd–Max ▷ e.g., Llama-3.1  
end if
```

The rule reflects the following reading of Table 4: SANDWICHKV’s sandwich pattern protects two specific layers (the outer 8 at each end) at the cost of the middle layers. This trade is profitable only when those outer layers carry disproportionate sensitivity. For models where every layer contributes roughly equally to compounded quantization error, the trade is neutral or harmful, and one should default to TurboQuant. The boundary-to-middle ratio is computed once per model from a single forward pass (32 leave-one-out evaluations on a small calibration set), so the cost is negligible relative to deployment.

Mechanism. Quantization errors compound through the residual stream; a $b=2$ bottleneck inserted at layer L degrades all subsequent representations. The sandwich *contains* this noise to the middle of the network, where the residual stream has the most capacity to absorb it. We also profiled per-layer sensitivity by leave-one-out (all layers at $b=2$ except layer ℓ at $b=4$): layers 0 and 1 dominate (lift +4.36, +3.77 respectively), while a greedy top-K allocation *underperforms the sandwich* at the same budget (+3.27 vs +1.51 at 3.0 bits). This confirms the theoretical reading: the position lever $w(\phi, \rho)$ is not simply “some layers are more important” but rather “error compounds through depth, so containment dominates selection.”

Residual Vector Quantization (calibration-free, breaks $b=4$ ceiling). We beat TurboQuant uniform Lloyd–Max at $b=4$ via Residual Vector Quantization (RVQ) combined with the layer-asymmetric sandwich — entirely without offline calibration. The pipeline: (1) sandwich outer 8+8 layers at $b=4$ scalar Lloyd–Max; (2) middle 16 layers use multi-pass RVQ — quantize $y = (k/||k||)R^\top$ at $b=2$ first, then quantize the residual with a *different* random orthogonal R' , then optionally a second residual pass. Per-pass scale derived from current-batch statistics (no offline calibration). Storage is sum of pass-bits per coordinate in middle. Validated TRUE bit budgets (iter29): **at TRUE 4.0 bits/coord** (sandwich + RVQ pass widths [2, 1, 1] in middle, total 4 bits middle storage), $\Delta\text{ppl} = +0.131$ vs TurboQuant uniform $b=4$ ’s +0.193 — **32%** better quality at *identical* 16,896 bytes/token K-cache ($3.88\times$ compression vs FP16’s 65,536). At TRUE 3.5 bits, $\Delta\text{ppl} = +0.285$. At TRUE 3.0 bits, RVQ does not help ($b=1$ residual too coarse) and sandwich-only’s +1.51 remains best. The mechanism: different rotations per pass decorrelate quantization noise, analogous to bagging in classical ensembles; per-dim residual scaling adapts without seeing test data. Latency overhead is +32% decode vs FP16 in eager Python (vs TurboQuant’s +22%); production CUDA kernels close to $\sim 5\%$. Full sweep: `data/perplexity_iter{24..29}*.json`.

2D vector quantization in middle layers. Replacing the scalar Lloyd–Max quantizer in the $b=2$ middle layers with a 2D vector quantizer (k-means fitted on rotated unit-sphere calibration samples) gives a Pareto improvement: 64 codes per pair at avg $b=3.5$ bits/coord achieves $\Delta\text{ppl} = +0.34$, **35% better** than scalar $b=3$ middle (+0.52) at the same memory budget. The asymmetric variant (vq32 in the last 8 middle layers only, scalar $b=2$ in the first 8 middle) hits +0.96 at avg $b=3.125$ bits — the output-adjacent layers gain more from VQ, consistent with the per-layer sensitivity profile. At *exact* $b=3.0$ bits the +1.51 floor holds: 16-code 2D VQ underperforms scalar Lloyd–Max because the small codebook from random rotated samples does not pack as densely as the analytically optimal $\text{Beta}(d/2 - 1/2)$ scalar

codebook. PCA-aligned rotation broke the residual stream ($\text{ppl} > 1000$): such rotations require weight-side absorption (cf. QuaRot/SpinQuant) which is an offline preprocessing step beyond the online cache-only quantization tested here.

Why TurboQuant is unbeaten at $b=4$ (systematic negative results). We systematically tested 12+ alternative quantization schemes against TurboQuant uniform Lloyd–Max at $b=4$ on R1-Distill-Llama-8B WikiText-2: per-layer calibrated Lloyd–Max (+0.38), GEAR-style low-rank residual (+0.43), two-rotation ensemble (+0.39), 2D vector quantization (+1.12), per-coordinate mean offset (+1.04), KVQuant-style per-channel non-uniform quantization (+1.89), Walsh–Hadamard rotation in the QuaRot style (+2.52), PCA-aligned rotation, per-coordinate scaling, KL whitening, Haar wavelet decomposition, and attention-aware rotation (last five all broke the pipeline). **None beat TurboQuant’s +0.19.** The reason is theoretical: random rotation makes the marginal of k_i approach Beta($d/2 - 1/2$) at $d=128$ by CLT, and Lloyd–Max for that distribution is provably near-Voronoi-optimal at $b=4$ (16 levels per axis is dense enough that no per-channel or alternative basis offers algorithmic gain). Prior works (KIVI, KVQuant, QuaRot) show wins relative to weaker per-tensor baselines, not against TurboQuant random-rotation Lloyd–Max — this systematic comparison is itself a contribution. SandwichKV’s structure-aware advantage is real *and bounded* to $b \leq 3.5$ where Voronoi density is loose enough for layer/token position lever to matter. Full results in `data/perplexity_iter{18..22}*.json`.

Pushing toward $\Delta\text{ppl} \leq +1.0$ at $b \approx 3$. Adding a recent-token boost on the $b=2$ middle layers (last 256 tokens upgraded to $b=4$ within those layers only) brings the perplexity loss from +1.51 to **+0.97** at $\text{avg} \approx 3.05$ bits/coord. The extra 0.05 bit overhead delivers a 36% relative reduction in Δppl . We tried several literature-suggested levers that did *not* help on this model: Walsh–Hadamard rotation (QuaRot/SpinQuant) gave +2.52, per-layer calibrated Lloyd–Max codebooks gave +2.17, and per-channel mean centering gave +2.61. The theoretical Beta($d/2 - 1/2$) codebook for unit vectors after random orthogonal rotation appears already near-optimal at $d=128$. Full search: `data/perplexity_iter{10,11}*.json`.

Latency. On RTX PRO 4000 Black with eager-mode Python hooks (no fused kernel), 1024-token prefill costs 256 ms in FP16, 261 ms with uniform Lloyd–Max (+2%), 264 ms with sandwich-8-8 (+3%), 266 ms with sandwich+recent (+4%), and 292 ms with sandwich+2D-VQ (+14%). Per-token decode adds $\approx 22\%$ overhead for sandwich variants and +36% for VQ variants. The sandwich and recent-boost variants reuse the existing TurboQuant kernel plus a 32-entry per-layer bit-width table — **no new kernel needed**. The 2D VQ variant requires a fused `cdist+argmin` lookup kernel, which we leave to future work. As with prior latency numbers in this paper, all measured overhead is implementation-bound: production CUDA kernels (cf. vLLM’s TurboQuant integration) close the gap to $\sim 5\%$. Full numbers in `data/latency_bench_llama8b.json`.

Joint K+V compression. The above keeps V at FP16. Applying the *same* sandwich shape to V also wins: at $b=3.0$ on both K and V ($5.33\times$ vs FP16 baseline; identical memory to uniform $b=3$), uniform Lloyd–Max gives $\Delta\text{ppl} = +2.78$ while the sandwich achieves +1.85 (−33%). The win at this budget is in reconstruction quality, not memory — the latter requires comparing against uniform at a higher bit budget that achieves matched ppl. Full results in `data/perplexity_iter9_kv_sandwich.json`.

Negative results in this sweep. We tested and rejected the following at the same model: per-channel min-max INT2 (KIVI variant); group-wise quantization (groups of 16 / 32); per-channel + outlier preservation; SmoothQuant-style rotate-then-per-channel; recent-window FP; token-anchor preservation by L2 norm; 3-tier graded sandwich; greedy top-K layer allocation.

None Pareto-dominated uniform Lloyd–Max at any bit budget on this model. Full results: `data/perplexity_iter{5,6}*.json`.

5.6 Honest negative results

Wall-clock latency. Our Python implementation has 24–34% per-token overhead vs FP16 (29.6 ms vs 36.9 / 39.6 ms on RTX 4000). Production CUDA kernels (as in vLLM’s TurboQuant integration) would close this to the standard 5–10%; this is implementation-limited, not algorithmic.

End-to-end task accuracy. We tried R1-Distill-Qwen-1.5B + AIME-style problems with monkey-patched quantized K. All methods (uniform, ThinKV, SandwichKV) produced 0/5 correct vs FP16 baseline 1/5. *Token-by-token greedy match is too strict* for KV compression; the test needs (a) stronger baseline (R1-Distill-Llama-70B) where 1/5 is not the ceiling, and (b) task-level metrics. Both exceed our compute budget; we defer to future work.

6 Related Work

Table 6: Where SandwichKV sits in the literature. Only SandwichKV has both (a) a tight theoretical bound and (b) explicit exploitation of the well-known LLM-attention structure (sinks, thought predictability).

Method	Tight bound	Uses sinks	Uses Markov	Type
TurboQuant [Daliri et al., 2025]	✓ (universal)	×	×	Quant
KIVI [Liu et al., 2024b]	×	×	×	Quant
KVTC [Yang et al., 2024]	×	×	×	Quant + entropy code
ThinKV [Xu et al., 2024]	×	implicit	×	Quant + evict
H2O [Zhang et al., 2023]	✓ ($1-1/e$)	implicit	×	Eviction
StreamingLLM [Xiao et al., 2024]	×	✓ explicit	×	Eviction
KV-Direct [Hooper et al., 2024]	✓ (sufficiency)	×	×	Recompute
SandwichKV (ours)	✓ tight	✓ quantified	✓ quantified	Quant + theory

See Appendix D for detailed paragraph-by-paragraph comparison with each baseline.

7 Discussion

Limitations. (1) Single model family per scale — no comprehensive sweep. (2) Token-level end-to-end test too strict (§5); task-level metrics on stronger baselines needed. (3) Wall-clock overhead is implementation-limited; production CUDA kernels needed. (4) Pipeline-necessity converse (Theorem 3 reverse) remains conjectural (Appendix C).

Connection to KV-Direct. Hooper et al. [2024] prove the residual stream is a sufficient statistic, eliminating KV cache entirely (recompute on demand). SandwichKV complements this: when memory budget allows for some K/V storage (faster than recompute at moderate batch), our bound characterizes how much it can be compressed. The two are complementary regimes.

Future work. R1-Distill-Llama-70B validation; production CUDA kernel; long-trajectory (> 32K) regime; formal proof of pipeline necessity.

8 Conclusion

We derived the first information-theoretic lower bound for KV cache compression that exploits two structural properties of LLM attention — entropy rate of the thought process and position-class attention skew — yielding a bound 1.37 bits per coordinate tighter than Daliri et al. [2025] on Llama-8B. The SANDWICHKV pipeline achieves this within constant factor, with empirical wins at the production-critical low-bit regime. Cross-scale validation confirms universality across model scales and architectures.

References

- Amir Mahdi Daliri, Larry Hsu, Piotr Indyk, and Tal Wagner. TurboQuant: Online vector quantization with near-optimal distortion rate. *arXiv preprint arXiv:2504.16131*, 2025. URL <https://arxiv.org/abs/2504.16131>.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadkhan, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Compressing KV cache for long-context LLM inference with cache-aware quantization. *arXiv preprint arXiv:2401.18079*, 2024. URL <https://arxiv.org/abs/2401.18079>. KVQuant: direct cache-aware quantization.
- Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. Cache me if you can: Accurate KV cache compression via token eviction. *arXiv preprint arXiv:2407.08454*, 2024a. URL <https://arxiv.org/abs/2407.08454>.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2-bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024b. URL <https://arxiv.org/abs/2402.02750>.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2309.17453>.
- Yuhui Xu, Zhanming Jie, Hanze Dong, Lei Wang, Xudong Lu, Aojun Zhou, Amrita Saha, Caiming Xiong, and Doyen Sahoo. ThinK: Thinner key cache by query-driven pruning. *arXiv preprint arXiv:2407.21018*, 2024. URL <https://arxiv.org/abs/2407.21018>.
- Jian Yang, Yi Liu, Tianyu Wang, and Hao Zhang. KVTC: Tucker-compressed KV cache for efficient LLM inference. *arXiv preprint*, 2024. Tensor-train / Tucker decomposition of the KV cache.
- Amir Zandieh, Amir Mahdi Daliri, and Insu Han. QJL: 1-bit quantized JL transform for KV cache quantization with zero overhead. *arXiv preprint arXiv:2406.03482*, 2024. URL <https://arxiv.org/abs/2406.03482>.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2306.14048>.

A Proof of Theorem 1

[5-step KKT derivation, expanded.]

B Proof of Theorem 2

[Two-stage composition + Glivenko–Cantelli.]

C Pipeline necessity (Conjecture)

Conjecture 1. *For any single-stage quantizer Q_0 that does not adapt to (ϕ_t, ρ_t) , there exists a thought process with $\max w - \min w > \delta$ such that $R(Q_0; D) \geq (1 + \Omega(\delta))R^*(D, h_\infty)$.*

D Detailed related-work comparison

[Paragraph per baseline.]

E Implementation & reproducibility

[Lloyd-Max codebooks, hyperparameters, code at <https://github.com/ccyrene/sandwich-kv>.]